DESENVOLVIMENTO DE CHATBOTS

Fundamentos teóricos e primeiros modelos

Objetivo da Aula
Compreender os diferentes modelos de chatbots existentes, suas arquiteturas, casos de uso e limitações.

PARTE 1 — FUNDAMENTOS TEÓRICOS

1.1 O que é um Chatbot?

Um chatbot é um sistema computacional projetado para simular conversas com humanos, seja por texto ou voz. Eles variam de simples sistemas baseados em regras até sofisticados modelos de linguagem natural treinados com bilhões de parâmetros.

Analogia Didática
Pense em chatbots como "atendentes virtuais" com diferentes níveis de inteligência:

- Nível 1 — Segue um roteiro fixo (script, como URA telefônica)
- Nível 2 — Entende palavras-chave e responde com padrões
- Nível 3 — Compreende linguagem natural e contexto
- Nível 4 — Raciocina, aprende e gera respostas originais (LLMs)

1.2 Taxonomia dos Modelos de Chatbots

Existem 5 grandes categorias de chatbots, cada uma com arquitetura, complexidade e casos de uso distintos:

Modelo	Tecnologia Base	Complexidade	Caso de Uso Principal
Rule-Based	Árvore de decisão / IF-ELSE	⭐	FAQs, menus, URA

Pattern Matching	Regex / AIML	⭐⭐	Atendimento simples
Retrieval-Based	TF-IDF / BM25 / Embeddings	⭐⭐⭐	Busca em base de conhecimento
Generative (seq2seq)	RNN / LSTM / Transformer	⭐⭐⭐⭐	Conversas abertas
LLM-Based	GPT, Claude, Gemini	⭐⭐⭐⭐⭐	Assistentes gerais, RAG

1.3 Modelo 1 — Rule-Based Chatbot

O modelo mais simples. Funciona com uma série de condições (IF-ELSE) ou fluxos de decisão pré-definidos. A resposta é sempre determinística: a mesma entrada sempre gera a mesma saída.

- Vantagens: simples de implementar, resultado previsível, sem custo de treinamento
- Desvantagens: frágil com variações de linguagem, não escala, não aprende
- Quando usar: fluxos simples e bem definidos (ex: "Selecione 1 para suporte, 2 para vendas")

1.4 Modelo 2 — Pattern Matching (AIML / Regex)

Utiliza expressões regulares (Regex) ou a linguagem AIML (Artificial Intelligence Markup Language) para associar padrões de entrada a respostas. É mais flexível que regras puras, mas ainda limitado.

- Vantagens: cobre variações simples de linguagem, legível e auditável
- Desvantagens: manutenção complexa com muitos padrões, não compreende contexto
- Quando usar: chatbots de suporte com perguntas frequentes limitadas

1.5 Modelo 3 — Retrieval-Based Chatbot

Em vez de gerar uma nova resposta, o bot busca a melhor resposta em uma base de dados de perguntas e respostas conhecidas. Utiliza técnicas de similaridade como TF-IDF, BM25 ou embeddings vetoriais.

- Vantagens: respostas sempre controladas e confiáveis, sem alucinações
- Desvantagens: limitado ao que está na base; não cria conteúdo novo
- Quando usar: helpdesks, documentação técnica, políticas internas

1.6 Modelo 4 — Generative Chatbot (Seq2Seq)

Baseado em redes neurais encoder-decoder (originalmente RNN/LSTM, hoje Transformers). O modelo aprende a gerar texto novo como resposta, não apenas selecionar de um banco. É o coração dos LLMs modernos.

- Vantagens: respostas criativas e contextuais, conversas abertas
- Desvantagens: pode alucinar, requer grande volume de dados e poder computacional
- Quando usar: assistentes gerais, tutores, entretenimento

1.7 Modelo 5 — LLM-Based Chatbot (via API)

Utiliza modelos de linguagem já pré-treinados (como Claude, GPT-4, Gemini) através de APIs. O desenvolvedor foca no design de prompts, lógica de negócio e integração — sem precisar treinar o modelo.

- Vantagens: alta qualidade imediata, fácil integração, raciocínio complexo
- Desvantagens: custo por token, dependência de API externa, privacidade de dados
- Quando usar: aplicações empresariais modernas, RAG, agentes inteligentes

1.8 Arquitetura Geral de um Chatbot

Independente do modelo, todo chatbot possui os seguintes componentes fundamentais:

1. Interface de Entrada: canal pelo qual o usuário envia a mensagem (texto, voz, API)
2. Processamento de Linguagem Natural (NLP): análise da intenção e entidades na mensagem
3. Gerenciamento de Diálogo: lógica de qual resposta gerar, com base no contexto
4. Geração ou Seleção de Resposta: criar ou buscar a resposta adequada
5. Interface de Saída: entrega da resposta ao usuário

Conceitos-Chave que Você Deve Conhecer

Intent (Intenção): o que o usuário quer fazer → Ex: 'quero comprar um produto'
Entity (Entidade): dados específicos na mensagem → Ex: 'produto = tênis, tamanho = 42'
Context (Contexto): histórico da conversa que influencia a próxima resposta
Utterance: cada frase enviada pelo usuário na conversa
Turn: uma rodada completa de pergunta + resposta

1.9 Formatos de Implementação em Python

Em Python, os chatbots podem ser implementados em diferentes formatos segundo sua complexidade:

Formato	Bibliotecas Principais	Adequado Para
Script puro	Python stdlib (input/print)	Protótipos, exercícios didáticos
Função / Módulo	Funções Python	Lógica reutilizável e testável
API REST	Flask, FastAPI	Integração com frontends e sistemas
Framework de Bot	ChatterBot, Rasa, Botpress	Projetos completos com NLU
API de LLM	anthropic, openai, google-generativeai	Chatbots com IA de ponta

PARTE 2 — EXERCÍCIOS PRÁTICOS EM PYTHON

Esta seção contém 6 exercícios independentes. Os três primeiros são códigos completos para você ler, executar e compreender. Os três últimos são desafios parciais onde você deverá completar o código, executar e analisar os resultados.

Pré-requisitos de Ambiente
Python 3.8+ | pip install anthropic difflib
Os exercícios 1, 2 e 3 não requerem bibliotecas externas.
Os exercícios 4, 5 e 6 indicam os imports necessários.

EXERCÍCIO 1 — LEITURA E COMPREENSÃO
Chatbot Rule-Based — Atendente de Lanchonete

Leia o código abaixo com atenção. Este é o modelo mais simples de chatbot: baseado em condições IF-ELSE. Ele simula um atendente de lanchonete que segue um fluxo fixo e determinístico.

```
exercicio_01_rule_based.py
# ============================================================
# EXERCÍCIO 1 — Chatbot Rule-Based (IF-ELSE)
# Modelo: Atendente de Lanchonete
# ============================================================

def chatbot_lanchonete():
```

```
    print('=== Bem-vindo à Lanchonete Digital! ===')
    print('Como posso ajudar você hoje?')
    print('Opções: [cardapio] [pedido] [preco] [sair]')
    print()

    cardapio = {
        'hamburguer': 25.90,
        'pizza':      18.50,
        'suco':        8.00,
        'sorvete':    12.00
    }

    while True:
        entrada = input('Você: ').strip().lower()

        if entrada == 'cardapio':
            print('Bot: Nosso cardápio:')
            for item, preco in cardapio.items():
                print(f'       {item.capitalize()} — R$ {preco:.2f}')

        elif entrada == 'pedido':
            item = input('Bot: Qual item você deseja? ').strip().lower()
            if item in cardapio:
                print(f'Bot: Perfeito! {item.capitalize()} adicionado. Total:
R$ {cardapio[item]:.2f}')
            else:
                print('Bot: Desculpe, esse item não está no cardápio.')

        elif entrada == 'preco':
            item = input('Bot: Preço de qual item? ').strip().lower()
            if item in cardapio:
                print(f'Bot: {item.capitalize()} custa R$
{cardapio[item]:.2f}.')
            else:
                print('Bot: Item não encontrado no cardápio.')

        elif entrada == 'sair':
            print('Bot: Obrigado pela visita! Até logo! 👋')
            break

        else:
            print('Bot: Não entendi. Tente: [cardapio] [pedido] [preco]
[sair]')

  # Executar o chatbot
  chatbot_lanchonete()
```

O que observar ao executar:

1. O bot só responde a exatamente as palavras-chave definidas.
2. Digite 'Cardápio' (com maiúscula) — o bot vai responder? Por quê?
3. O que acontece se você digitar uma frase completa como 'quero ver o cardápio'?
4. Como o .strip().lower() contribui para a robustez do sistema?
5. Identifique: qual é o 'fluxo de diálogo' deste bot? Desenhe-o.

EXERCÍCIO 2 — LEITURA E COMPREENSÃO
Chatbot Pattern Matching — Suporte com Regex

Este chatbot usa expressões regulares (Regex) para identificar padrões nas mensagens do usuário. É mais flexível que IF-ELSE: responde a variações como 'oi', 'olá', 'boa tarde' com a mesma regra.

```
exercicio_02_pattern_matching.py
# ============================================================
# EXERCÍCIO 2 — Chatbot Pattern Matching com Regex
# Modelo: Suporte técnico básico
# ============================================================
import re

# Base de padrões: (padrão_regex, resposta)
PADROES = [
    (r'\b(oi|olá|ola|bom dia|boa tarde|boa noite|hey|hello)\b',
     'Olá! Bem-vindo ao suporte técnico. Como posso ajudar?'),

    (r'\b(senha|password|login|acesso)\b',
     'Para redefinir sua senha, acesse: configuracoes > seguranca > redefinir
senha.'),

    (r'\b(lento|travando|devagar|lag|performance)\b',
     'Tente limpar o cache do navegador e reiniciar o sistema.'),

    (r'\b(erro|error|bug|falha|problema)\b',
     'Pode descrever o erro com mais detalhes? Qual mensagem aparece na
tela?'),

    (r'\b(obrigad[ao]|valeu|thanks|thank you)\b',
     'Fico feliz em ajudar! Há mais alguma coisa?'),

    (r'\b(tchau|sair|encerrar|bye|adeus)\b',
     'ENCERRAR'),
]

def responder(mensagem: str) -> str:
    mensagem_lower = mensagem.lower()
    for padrao, resposta in PADROES:
        if re.search(padrao, mensagem_lower):
            return resposta
    return 'Não encontrei informações sobre isso. Poderia reformular a
pergunta?'

def chatbot_suporte():
    print('=== Chatbot de Suporte Técnico v2.0 ===')
    print('Digite sua dúvida ou "tchau" para sair.')
    print()
    while True:
        entrada = input('Você: ').strip()
        if not entrada:
            continue
```

```
        resposta = responder(entrada)
        if resposta == 'ENCERRAR':
            print('Bot: Até logo! Chamado encerrado. 🎫')
            break
        print(f'Bot: {resposta}')
        print()

chatbot_suporte()
```

O que observar ao executar:

1. Digite 'Boa tarde!' — o bot responde ao cumprimento mesmo com maiúscula e '!'?
2. Digite 'Meu login não funciona' — quais padrões são ativados?
3. Digite 'Estou com problema de login lento' — o bot captura ambos os padrões? Qual vence?
4. O que significa o \b no Regex? Pesquise e explique com suas palavras.
5. Adicione um novo padrão para responder sobre 'fatura' ou 'pagamento'.

EXERCÍCIO 3 — LEITURA E COMPREENSÃO
Chatbot Retrieval-Based — FAQ com Similaridade de Texto

Este modelo não segue regras fixas. Em vez disso, ele busca a pergunta mais similar na base de conhecimento usando o algoritmo SequenceMatcher (similaridade de strings). É a fundação dos sistemas de busca modernos.

```
exercicio_03_retrieval_based.py
# ============================================================
# EXERCÍCIO 3 — Chatbot Retrieval-Based
# Modelo: FAQ com similaridade de strings (difflib)
# ============================================================
from difflib import SequenceMatcher

# Base de conhecimento: perguntas e respostas
FAQ = {
    'como faço para criar uma conta':   'Acesse o site, clique em Cadastre-se
e preencha o formulário.',
    'qual o prazo de entrega':          'O prazo padrão é de 5 a 10 dias
úteis.',
    'como rastrear meu pedido':         'Acesse Minha Conta > Pedidos >
Rastrear.',
    'posso trocar um produto':           'Sim! Trocas são aceitas em até 30
dias com nota fiscal.',
    'como cancelar minha compra':       'Pedidos podem ser cancelados em até
24h após a compra.',
    'quais formas de pagamento':        'Aceitamos cartão, Pix e boleto
bancário.',
    'tem frete gratis':                 'Frete grátis para compras acima de
R$ 150,00.',
    'como falar com atendente humano':  'Digite 0 a qualquer momento para
transferir ao atendente.',
```

```
}

def similaridade(a: str, b: str) -> float:
    return SequenceMatcher(None, a.lower(), b.lower()).ratio()

def buscar_resposta(pergunta: str, limiar: float = 0.45) -> str:
    melhor_score = 0.0
    melhor_resposta = None

    for faq_pergunta, faq_resposta in FAQ.items():
        score = similaridade(pergunta, faq_pergunta)
        if score > melhor_score:
            melhor_score = score
            melhor_resposta = faq_resposta

    print(f'  [DEBUG] Melhor score de similaridade: {melhor_score:.2f}')

    if melhor_score >= limiar:
        return melhor_resposta
    return 'Desculpe, não encontrei uma resposta para isso. Tente
reformular.'

def chatbot_faq():
    print('=== Central de Atendimento — FAQ Inteligente ===')
    print('Faça sua pergunta ou digite "sair".')
    print()
    while True:
        pergunta = input('Você: ').strip()
        if pergunta.lower() in ('sair', 'tchau', 'bye'):
            print('Bot: Atendimento encerrado. Obrigado!')
            break
        resposta = buscar_resposta(pergunta)
        print(f'Bot: {resposta}')
        print()

chatbot_faq()
```

O que observar ao executar:

1. Pergunte 'como criar conta' — o bot encontra a resposta correta? Qual foi o score?
2. Pergunte 'tem desconto no frete' — o bot retorna algo útil? Por quê?
3. Altere o limiar (limiar=0.45) para 0.80 e refaça as perguntas. O que muda?
4. Qual é o trade-off entre um limiar muito baixo vs muito alto?
5. O que o modo DEBUG (linha com [DEBUG]) revela sobre como o algoritmo funciona?

EXERCÍCIO 4 — DESAFIO PRÁTICO
Chatbot com Memória de Contexto — Multi-Turn

Um chatbot só é realmente útil quando lembra o que foi dito antes na conversa. Este exercício implementa um chatbot simples com histórico de contexto. O código está incompleto — você deve preencher as partes indicadas.

O QUE VOCÊ DEVE FAZER:

1. Preencha as 3 funções marcadas com TODO no código abaixo.
2. Execute o chatbot e mantenha uma conversa de pelo menos 5 turnos.
3. Observe o histórico sendo construído a cada turno.
4. Responda: o que acontece se o histórico crescer indefinidamente em um sistema real? Como mitigar?

```
exercicio_04_contexto_INCOMPLETO.py
# ============================================================
# EXERCÍCIO 4 — Chatbot com Memória de Contexto
# Você deve completar as funções marcadas com TODO
# ============================================================

from datetime import datetime

# Estrutura do histórico de conversa
historico = []  # Lista de dicionários {'turno': N, 'usuario': '...', 'bot':
'...'}

RESPOSTAS = {
    'nome':     'Meu nome é ByteBot, seu assistente digital!',
    'ajuda':    'Posso responder sobre: nome, turno, historico, hora.',
    'hora':     lambda: f'Agora são {datetime.now().strftime("%H:%M:%S")}.',
}

def processar_mensagem(mensagem: str, turno: int) -> str:
    msg = mensagem.lower().strip()

    if 'nome' in msg:
        return RESPOSTAS['nome']
    elif 'hora' in msg:
        return RESPOSTAS['hora']()
    elif 'turno' in msg:
        # TODO: retorne quantos turnos já ocorreram
        pass
    elif 'historico' in msg or 'histórico' in msg:
        # TODO: retorne um resumo do histórico (últimas 3 mensagens do
usuário)
        pass
    else:
        return 'Não entendi. Digite "ajuda" para ver o que posso fazer.'

def registrar_turno(turno: int, usuario: str, bot: str):
    # TODO: adicione o turno ao histórico com os campos corretos
    pass

def chatbot_contextual():
    print('=== ByteBot — Chatbot com Memória ===')
    turno = 0
    while True:
        entrada = input('Você: ').strip()
        if entrada.lower() in ('sair', 'tchau'):
            print('Bot: Até mais! Foram', turno, 'turnos de conversa.')
            break
        turno += 1
```

```
        resposta = processar_mensagem(entrada, turno)
        registrar_turno(turno, entrada, resposta)
        print(f'Bot: {resposta}')
        print()

chatbot_contextual()
```

EXERCÍCIO 5 — DESAFIO PRÁTICO
Chatbot com Intenções e Entidades — Pedido de Pizza

Neste exercício, você vai construir um chatbot que extrai intenção (o que o usuário quer) e entidades (os dados específicos). O esqueleto do código está pronto; você deve implementar o extrator de entidades e a lógica de pedido.

O QUE VOCÊ DEVE FAZER:

1. Implemente a função extrair_entidades() que identifica sabor e tamanho da pizza na frase.
2. Implemente a função confirmar_pedido() que formata e exibe o pedido final.
3. Teste com frases como: 'quero uma pizza grande de calabresa' e 'pizza média de frango'.
4. Avalie: o bot consegue lidar com 'uma pizzinha pequenininha de queijo'? Por que não? O que seria necessário para isso?

```
exercicio_05_intencao_entidade_INCOMPLETO.py
# ============================================================
# EXERCÍCIO 5 — Intent + Entity Extraction
# Chatbot de Pedido de Pizza
# ============================================================
import re

SABORES   = ['calabresa', 'frango', 'queijo', 'portuguesa', 'vegetariana',
'pepperoni']
TAMANHOS  = ['pequena', 'media', 'média', 'grande', 'gigante', 'família',
'familia']

def detectar_intencao(mensagem: str) -> str:
    msg = mensagem.lower()
    if any(p in msg for p in ['quero', 'pedir', 'pedido', 'comprar',
'queria']):
        return 'FAZER_PEDIDO'
    elif any(p in msg for p in ['cancelar', 'desistir', 'não quero']):
        return 'CANCELAR'
    elif any(p in msg for p in ['cardápio', 'cardapio', 'opções', 'opcoes',
'tem']):
        return 'VER_CARDAPIO'
    return 'DESCONHECIDO'

def extrair_entidades(mensagem: str) -> dict:
    # TODO: use as listas SABORES e TAMANHOS para identificar
    # o sabor e tamanho mencionados na mensagem.
```

```
    # Retorne um dicionário: {'sabor': '...' ou None, 'tamanho': '...' ou
None}
    pass

def confirmar_pedido(entidades: dict) -> str:
    # TODO: verifique se sabor e tamanho foram encontrados.
    # Se sim, retorne uma mensagem de confirmação formatada.
    # Se algum estiver faltando, pergunte ao usuário.
    pass

def chatbot_pizza():
    print('=== PizzaBot — Faça seu pedido! ===')
    while True:
        entrada = input('Você: ').strip()
        if entrada.lower() in ('sair', 'tchau'):
            print('Bot: Pedido cancelado. Até logo!')
            break

        intencao = detectar_intencao(entrada)
        print(f'  [DEBUG] Intenção: {intencao}')

        if intencao == 'VER_CARDAPIO':
            print(f'Bot: Sabores disponíveis: {", ".join(SABORES)}')
            print(f'Bot: Tamanhos: {", ".join(["Pequena", "Média",
"Grande"])}')
        elif intencao == 'FAZER_PEDIDO':
            entidades = extrair_entidades(entrada)
            print(f'  [DEBUG] Entidades: {entidades}')
            resposta = confirmar_pedido(entidades)
            print(f'Bot: {resposta}')
        elif intencao == 'CANCELAR':
            print('Bot: Seu pedido foi cancelado.')
        else:
            print('Bot: Não entendi. Você pode pedir uma pizza ou ver o
cardápio.')
        print()

chatbot_pizza()
```

EXERCÍCIO 6 — DESAFIO PRÁTICO

Chatbot Multi-Personalidade — Troca de Comportamento em Runtime

Este é o exercício mais avançado da aula. Você vai implementar um chatbot capaz de alternar entre diferentes 'personalidades' (modos de comportamento) em tempo real, durante a conversa. Este padrão é a base dos sistemas de prompt-switching usados em LLMs.

O QUE VOCÊ DEVE FAZER:

1. Implemente a classe Personalidade com os atributos: nome, tom, prefixo_resposta e vocabulario_proibido.
2. Implemente o método chatbot.trocar_personalidade(nome) que altera a personalidade ativa.

3. Implemente chatbot.gerar_resposta() que aplica o tom e filtra vocabulário proibido.
4. Teste trocando de 'formal' para 'casual' no meio da conversa e observe as diferenças.
5. Reflexão escrita: como este padrão se relaciona com o conceito de 'System Prompt' em LLMs como Claude?

```
exercicio_06_multi_personalidade_INCOMPLETO.py
# ============================================================
# EXERCÍCIO 6 — Chatbot Multi-Personalidade
# Padrão avançado: troca de comportamento em runtime
# ============================================================

class Personalidade:
    def __init__(self, nome, tom, prefixo, vocab_proibido):
        # TODO: inicialize os atributos
        pass

# Defina pelo menos 3 personalidades:
PERSONALIDADES = {
    'formal': Personalidade(
        nome='Assistente Formal',
        tom='cordial e profissional',
        prefixo='Prezado usuário, ',
        vocab_proibido=['cara', 'mano', 'oi', 'beleza', 'vlw']
    ),
    'casual': Personalidade(
        nome='ChatZinho',
        tom='descontraído e jovem',
        prefixo='Oi! ',
        vocab_proibido=['prezado', 'solicito', 'outrossim', 'conforme']
    ),
    'tecnico': Personalidade(
        nome='TechBot',
        tom='técnico e direto',
        prefixo='RESPOSTA: ',
        vocab_proibido=['acho', 'talvez', 'quem sabe']
    ),
}

RESPOSTAS_GENERICAS = {
    'python':    'Python é uma linguagem de programação interpretada e de
alto nível.',
    'chatbot':   'Chatbot é um sistema que simula conversas humanas de forma
automatizada.',
    'ia':        'Inteligência Artificial é a capacidade de máquinas
realizarem tarefas cognitivas.',
    'ajuda':     'Posso falar sobre: python, chatbot, ia. Troque
personalidade com /modo.',
}

class ChatbotMultiPersonalidade:
    def __init__(self):
        self.personalidade_ativa = PERSONALIDADES['formal']
        self.historico = []

    def trocar_personalidade(self, nome: str) -> str:
        # TODO: troque a personalidade e retorne mensagem de confirmação
```

```
        pass

    def gerar_resposta(self, mensagem: str) -> str:
        # TODO:
        # 1. Busque a resposta base em RESPOSTAS_GENERICAS
        # 2. Aplique o prefixo da personalidade ativa
        # 3. Filtre vocabulário proibido (substitua por '***')
        # 4. Retorne a resposta formatada
        pass

    def executar(self):
        print('=== Chatbot Multi-Personalidade ===')
        print('Comandos: /modo formal | /modo casual | /modo tecnico | sair')
        print(f'Personalidade ativa: {self.personalidade_ativa.nome}')
        print()
        while True:
            entrada = input('Você: ').strip()
            if entrada.lower() == 'sair':
                print('Encerrando. Até logo!')
                break
            elif entrada.lower().startswith('/modo '):
                nome = entrada.split(' ', 1)[1].lower()
                print(f'Bot: {self.trocar_personalidade(nome)}')
            else:
                resposta = self.gerar_resposta(entrada)
                print(f'Bot: {resposta}')
            print()

bot = ChatbotMultiPersonalidade()
bot.executar()
```

Critérios de Avaliação dos Desafios:

- O código executa sem erros
- A lógica implementada está correta e coerente
- Os outputs observados foram documentados (prints ou comentários)
- As perguntas de reflexão foram respondidas